

DocMind RAG

文档检索增强生成知识库服务

项目设计与实施说明书

聚焦 RAG 工程闭环，不堆叠 Agent / GraphRAG / 多模型编排
文档摄取 · 切分 · Embedding · 向量检索 · 来源引用 · 拒答 · 评估 · 测试 · 容器化
目标：可运行、可评估、可解释、可复现、可定位失败

0. 本版本定稿原则

本说明书作为 DocMind RAG 的固定实施范围。保留初版“文档摄取—检索—生成—来源引用—评估”的主线，同时去掉不影响项目完整度的 Query Rewrite、Rerank、混合检索、Ragas、Agent、GraphRAG 等增强项。所有保留能力均要求能够在代码、测试、运行结果或固定实验中被验证。

调整项	定稿处理	目的
文档格式	固定为文本型 PDF + UTF-8 Markdown	覆盖两类常见技术资料，同时避免 OCR、多模态解析扩大范围
向量存储	固定使用 Chroma 持久化模式	同时满足 metadata、检索和按 document_id 删除，不引入独立向量数据库服务
文档元数据	固定使用 SQLite	保存 document/checksum/chunk_count 等生命周期信息，不把业务元数据完全寄托在向量库
RAG 流程	固定为 2-step RAG	Question → Retrieve → Context → LLM，不加入 Agent 或多阶段编排
检索优化	仅保留 chunk_size 与 Top-K 对照实验	用少量可重复实验体现优化意识，不堆叠 Rewrite/Rerank/Hybrid
拒答	加入检索相关性门控 + Prompt 约束	避免“向量库总会返回 Top-K”导致无关证据也被强行送入 LLM
来源引用	采用受控 source label 并校验	模型只能引用本次上下文中的 S1/S2...，未知来源标签不被信任
测试 / CI	云端模型通过 Client 抽象并在 CI 中使用 fake/mock	保证自动化测试稳定，不依赖真实 API Key、网络或调用费用
健康检查	拆分 /health/live 与 /health/ready	ready 检查本地存储与必要配置，不周期性调用收费 LLM API

目录

- 1. 项目定义与目标
- 2. 业务边界与固定范围
- 3. 核心对象与业务规则
- 4. 功能需求与验收标准

- 5. 非功能要求
- 6. 总体架构与分层职责
- 7. 技术组件与选型原则
- 8. 数据与存储设计
- 9. 文档摄取与一致性设计
- 10. 检索、生成与来源引用
- 11. API 设计规范
- 12. 评估设计
- 13. 测试设计
- 14. 日志、健康检查与可观测性
- 15. 运行、容器化与 CI
- 16. 开发阶段与完成定义
- 17. 固定边界与设计取舍
- 18. 最终验收清单

1. 项目定义与目标

DocMind RAG 是面向技术文档、课程笔记和产品手册等资料的单用户文档知识库 API。用户上传资料后，系统完成校验、解析、清洗、切分、Embedding 和向量索引；提问时先检索相关证据，再让 LLM 仅基于证据生成回答，并返回可定位到原文的来源信息。

项目核心定位

- 不是“上传 PDF 后调用一次 LLM”，而是实现可拆分、可调试、可评估的 RAG pipeline。
- 检索质量与生成质量分开分析：先确认正确证据是否召回，再判断模型是否正确利用证据。
- 所有关键参数和实验结果必须可记录、可重复，不以单个演示问题的主观效果作为完成标准。
- 不宣称消除幻觉；通过证据门控、受控引用、拒答和评估降低无依据回答。

类别	内容
核心目标	稳定完成文档摄取、向量检索、基于证据回答、来源引用与拒答。
工程目标	FastAPI 服务可测试、可容器化运行；本地持久化元数据和向量；外部模型调用具备超时与错误处理。
评估目标	使用固定问题集评估 Recall@K、无答案拒答准确率与延迟，并记录 chunk/Top-K 对照结果。
展示目标	仓库可由陌生人按 README 启动；样例文档、评测集和实验结果能够复现主要结论。
运行假设	单用户、单实例、私有/本地使用；云端 LLM 与 Embedding API；不面向公网多租户服务。

2. 业务边界与固定范围

范围类型	范围	说明
固定范围	PDF/Markdown 上传、解析、切分、Embedding、Chroma 索引、文档列表/详情/删除、/retrieve、/ask、sources、拒答、固定评测、pytest、日志、Docker Compose、CI	全部必须完成，并进入最终验收。
明确不包含	OCR、多模态解析、用户登录/权限、多知识库、Query Rewrite、Rerank、BM25/Hybrid、Ragas、Agent、GraphRAG、模型微调、异步任务、分布式向量库	不是待办项；本项目完成后不以这些能力作为“补全项目”的条件。

- PDF 仅支持可直接提取文本的 PDF；扫描件、纯图片 PDF、加密且不可解析的 PDF 按不支持输入处理。

- Markdown 文件统一按 UTF-8 读取；不承担富文本、Office 文档或网页抓取。
- 原始上传文件只用于本次解析，使用服务端生成的临时文件名；索引完成后删除临时文件，避免把用户原始文件长期落盘。
- 公开仓库仅提供具有公开权限的样例资料；API Key、私人文档、真实评测数据不提交仓库。

3. 核心对象与业务规则

对象	能力 / 字段	关键规则
Document	document_id、filename、file_type、checksum、chunk_count、created_at	只保存已成功建立索引的文档元数据；checksum 唯一，重复内容返回 409。
Chunk	chunk_id、document_id、text、page/section、chunk_index	存入 Chroma；必须保留 document_id 与来源定位信息；删除文档时按 document_id 删除全部 chunks。
Question	question	不能为空；长度设上限；不允许客户端直接注入系统 Prompt 或伪造 source id。
RetrievedChunk	rank、distance、metadata、text	distance 仅表示当前向量检索配置下的相关性信号，不解释为“答案置信度”。
Source	source_id、document_id、filename、page/section、excerpt	source_id 为本次回答临时生成的 S1/S2...；只映射到本次实际送入上下文的证据。
QuestionCase	case_id、question、answerable、expected_source	评测集固定；expected_source 使用文档 + 页/章节定位，不绑定会随切分参数变化的 chunk_id。

最重要的业务规则

- 只有 metadata DB 中存在的 document_id 才被视为有效文档；检索阶段会忽略没有对应 Document 记录的孤儿向量。
- 无足够相关证据时直接返回固定拒答，不调用 LLM；“Top-K 总能返回结果”不等同于“存在答案”。
- LLM 不能决定真实来源。Context Builder 先分配 S1/S2...，模型只能在这些标签范围内引用，API 再映射到 filename/page/section。
- 文档删除成功的前提是向量删除与元数据删除均完成；任一步失败都不能返回 204。

4. 功能需求与验收标准

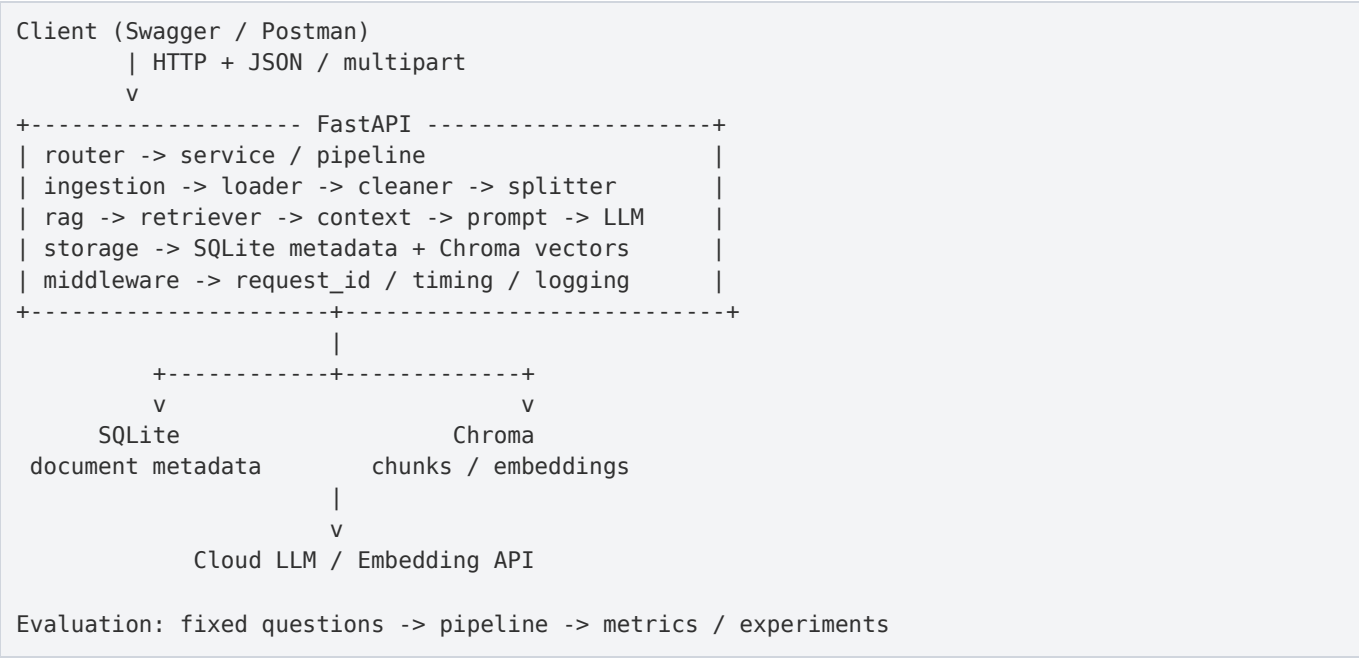
模块	需求	验收标准
文档上传	POST /documents；支持 .pdf / .md；最大 10 MB；计算 SHA-256 checksum	合法文档返回 201；重复内容 409；格式/大小/空文档/不可解析 PDF 返回 422。
解析与切分	PDF 保留 page；Markdown 保留 heading/section；统一清洗后切 chunk	每个 chunk 有稳定 document_id、chunk_index 与可定位来源；空 chunk 不入库。
Embedding / 索引	批量调用 Embedding Client，写入 Chroma 持久化集合	索引成功后才写 Document 元数据；chunk_count 与实际向量数量一致。
文档查询	GET /documents、GET /documents/{id}	只返回已索引成功文档；详情含 filename/type/chunk_count/created_at。
文档删除	DELETE /documents/{id}	先删除该 document_id 的全部向量，再删除元数据；成功 204；不存在 404。
纯检索	POST /retrieve	返回 Top-K chunks、rank、distance、来源 metadata；用于调试和评估。
RAG 问答	POST /ask	有证据时返回 answer + sources；无足够证据时 200 + 固定拒答，且不调用 LLM。

模块	需求	验收标准
来源引用	Context Builder 分配 S1/S2...并要求答案引用	sources 可映射回 filename + page/section + excerpt；未知 source label 不被接受。
评估	固定 30 个问题；运行 Baseline/Chunk/Top-K 三组配置	输出 Recall@K、拒答准确率、检索/LLM/总延迟与实验配置 CSV/JSON。
健康检查	/health/live 与 /health/ready	live 不依赖外部服务；ready 检查 SQLite、Chroma 路径和必要模型配置，异常返回 503。

5. 非功能要求

方面	要求	可验收标准
安全	API Key/Endpoint 从环境变量读取；日志不打印 API Key、Authorization、完整文档、完整 Prompt；临时上传文件使用服务端生成路径	仓库不含真实密钥或私人资料；日志抽查无敏感内容。
一致性	SQLite 与 Chroma 无分布式事务，采用明确的写入顺序、补偿删除和幂等 delete；检索忽略孤儿向量	模拟元数据写入失败时向量被清理；删除失败不返回成功。
可维护性	router / ingestion / rag / storage / clients / core 分层；外部模型 Client 抽象	路由层不直接处理向量库和 Prompt 细节；fake client 可替换真实 client。
可测试性	pytest；外部 LLM/Embedding 在自动化测试中 fake/mock；Chroma 使用临时目录	测试不依赖网络、API Key 或调用费用；可重复通过。
可观测性	request_id + ingestion/retrieval/LLM/total latency；异常阶段化记录	可通过日志判断失败发生在 parse、embed、retrieve、LLM 还是 storage。
可复现	README + .env.example + Docker Compose + 固定样例文档 + 固定评测集	新环境可启动服务、导入样例、执行 pytest 与 eval。
性能边界	单实例、小型知识库；Embedding 批处理；Context 有字符/token 预算；文件 10 MB 上限	不宣称高并发；主要优化对象是检索质量、外部模型延迟与上下文长度。

6. 总体架构与分层职责



层	职责	禁止事项
router	HTTP 协议、文件接收、依赖注入、请求/响应与状态码	不直接写 Chroma 查询，不拼 Prompt，不做文档解析。
ingestion	文件校验、解析、清洗、切分、Embedding、索引编排	不负责问答生成；解析器不直接写 API 响应。
rag	检索、证据门控、Context Builder、source label、Prompt、生成结果校验	不把 distance 当答案置信度；不信任模型生成的任意来源。
storage	DocumentRepository、VectorStoreAdapter、持久化与删除	调用方不直接依赖 Chroma/SQLite 具体实现。
clients	EmbeddingClient / LLMClient；超时、有限重试、错误转换	业务层不散落供应商 SDK 调用。
core	配置、日志、异常、request_id	不硬编码 API Key、模型名或数据目录。
eval	固定问题集、实验配置、指标与结果落盘	不在实验之间随意修改问题集以追求更好结果。

7. 技术组件与选型原则

组件	职责	项目中的使用方式
FastAPI	REST API / OpenAPI / 依赖注入	documents、retrieve、ask、health 接口；Swagger 作为主要交互入口。
Pydantic	请求/响应 schema 与参数校验	限制 question 长度、响应 sources 结构、错误响应结构。
SQLite	文档元数据	保存 Document；checksum 唯一；小型单用户服务无需独立数据库进程。
SQLAlchemy	SQLite 访问与事务	Repository 管理元数据写入/删除；避免业务层散落 SQL。
Chroma	本地持久化向量存储	单 collection；cosine 距离；chunk text + metadata 一起保存；按 document_id 删除。
PDF / Markdown Loader	文档解析	PDF 仅文本型；Markdown UTF-8；统一输出 text + source locator。
Embedding Client	文档/query → 向量	文档与 query 使用同一兼容 embedding 模型；支持批处理和 mock。

组件	职责	项目中的使用方式
LLM Client	基于证据生成答案	模型/Endpoint/API Key 配置化；固定超时；仅接收构建后的 context。
pytest + httpx	单元与 API 测试	Loader、Splitter、Storage、Retriever、Citation、API、失败补偿。
Docker Compose	可复现运行	单 api 容器 + SQLite/Chroma 持久化卷，不额外部署向量数据库服务。
GitHub Actions	持续集成	push / PR 安装依赖并运行 pytest；CI 不访问真实云端模型。

选型原则

- 组件必须服务于固定需求；本项目不为了“更像 AI 系统”而叠加 Agent、GraphRAG、Rerank、Hybrid Search。
- Chroma 与 SQLite 都采用本地持久化，是单用户项目的主动取舍，不包装为生产级分布式架构。

8. 数据与存储设计

8.1 SQLite : Document 元数据

字段	约束 / 类型	说明
document_id	TEXT / PK	服务端生成 UUID；同时写入每个 chunk metadata。
filename	TEXT / NOT NULL	仅保存显示用文件名，不直接作为服务器文件路径。
file_type	TEXT / CHECK(pdf, md)	固定两种格式。
checksum	TEXT / UNIQUE / NOT NULL	SHA-256；同一内容重复上传返回 409。
chunk_count	INTEGER / >= 1	索引完成后记录实际 chunk 数。
created_at	UTC datetime	统一存 UTC；API 返回 ISO 8601。

8.2 Chroma : Chunk / Embedding

内容	metadata	规则
chunk text + vector	chunk_id, document_id, filename, file_type, chunk_index	chunk_id 在一次索引中唯一；document_id 是删除与有效性校验关键字段。
PDF 来源	page	页码从 1 开始；PDF chunk 不跨页，优先保证引用可定位。
Markdown 来源	section	保留标题路径，如“3.2 / Configuration”；尽量在章节内切分。
检索字段	distance / rank	Chroma collection 固定 cosine 距离；distance 越小表示越相似，只作为检索信号。

8.3 评测数据

对象	字段	说明
QuestionCase	case_id, question, answerable, expected_document, expected_locator, reference_answer(optional)	expected_locator 使用页码/章节等稳定来源定位，不使用随 chunk 参数变化的 chunk_id。

对象	字段	说明
ExperimentResult	config_id, chunk_size, overlap, top_k, model names, metrics, latency, timestamp	每次实验保存完整配置和失败 case；不只保留最好结果。

9. 文档摄取与一致性设计

9.1 上传与索引流程

```
POST /documents
-> 校验扩展名 / MIME / 大小
-> 临时保存到服务端生成的随机路径
-> 计算 SHA-256；检查 checksum 是否重复
-> Parse -> Clean -> Chunk
-> 批量 Embedding
-> 写入 Chroma(document_id + metadata)
-> 写入 SQLite Document 并 commit
-> 删除临时文件
-> 返回 201
```

- SQLite 与 Chroma 之间不存在 ACID 分布式事务，因此实现中必须明确处理跨存储失败，而不是把两次写入描述成“一个事务”。
- Chroma 写入失败：不写 Document 元数据，清理临时文件并返回受控错误。
 - Chroma 已成功但 SQLite commit 失败：rollback SQLite，并按 document_id 对 Chroma 执行补偿删除；补偿失败必须记录 ERROR 日志。
 - 进程在两次写入之间异常退出可能留下孤儿向量；Retriever 在使用结果前校验 document_id 是否仍存在于 SQLite，孤儿向量不会进入 Context。
 - 同一 checksum 由数据库 UNIQUE 兜底；应用层预查只用于更友好提示，不能替代唯一约束。

9.2 Chunk 规则

规则	固定处理	原因
默认参数	chunk_size=700 字符, overlap=100 字符	作为基线，不宣称最优；最终由固定评测集验证。
PDF	每页独立清洗和切分，不跨 page 拼 chunk	来源页码稳定，引用更容易核验。
Markdown	先按标题层级形成 section，再对超长 section 切分	尽量保留结构语义，同时控制上下文长度。
清洗	统一换行/空白；去除空段；不做激进正文删除	避免复杂启发式清洗误删有效内容。
metadata	document/page or section/chunk_index 全链路继承	检索、Context、来源引用和评估都依赖这些字段。

9.3 删除流程

```
DELETE /documents/{id}
-> 查询 SQLite: 不存在则 404
-> Chroma delete where document_id = ?
-> 验证该 document_id 不再返回向量
-> 删除 SQLite Document + commit
-> 返回 204
```

向量删除失败时保留 Document 元数据并返回错误，避免 API 返回成功但旧知识仍可被检索。SQLite 删除失败则 rollback 并返回错误；再次调用 DELETE 应可安全重试。

10. 检索、生成与来源引用

10.1 Baseline RAG 流程

```
POST /ask
-> 校验 question
-> embed(question)
-> Chroma cosine search Top-K
-> 过滤无有效 Document 的孤儿 chunk
-> Evidence Gate
    |-- evidence insufficient -> 固定拒答（不调用 LLM）
    |-- evidence sufficient   -> Context Builder
                                -> S1/S2/... labels
                                -> Prompt + Question -> LLM
                                -> Citation validation
                                -> Answer + Sources
```

10.2 证据门控与拒答

- Chroma 即使面对完全无关的问题也可能返回 Top-K，因此“有返回结果”不能作为“有证据”的判定。
- collection 固定使用 cosine 距离。项目维护一个 max_distance 配置值；当无结果或最佳结果超过阈值时，直接返回“无法从当前资料确认”。
- max_distance 通过固定评测集中的可回答问题与无答案问题进行校准，并记录在实验配置中；它不是概率置信度。
- 拒答分支不调用 LLM，从而减少无依据生成，同时降低无效调用成本。

10.3 Context 与来源引用

- 按 rank 顺序选择证据，并设置最大 context 字符/token 预算，防止大量 Top-K 直接撑满 Prompt。
- Context Builder 为本次实际使用的 chunks 生成 S1、S2...；Prompt 要求事实陈述使用这些标签引用。
- 模型生成的标签必须属于当前允许集合。出现未知来源标签时，不映射到任何文档；结果按 MODEL_OUTPUT_INVALID 处理并记录 request_id。
- API sources 返回 document_id、filename、page/section、chunk_index 与限长 excerpt；不把检索 distance 表述为答案置信度。

10.4 Prompt 约束

- 只依据提供的 Context 回答，不使用未提供的事实补齐答案。
 - 资料不足时明确回答“无法从当前资料确认”。
 - 需要引用时仅使用 Context 中提供的 [S1]、[S2]... 标签。
 - 不把 source label、distance 或检索排名解释为模型置信度。

11. API 设计规范

方法	路径	功能	成功 / 主要异常
POST	/documents	上传并同步建立索引	201; 409 重复; 422 文件非法; 502 Embedding provider 错误
GET	/documents	文档列表	200
GET	/documents/{id}	文档详情	200 / 404
DELETE	/documents/{id}	删除向量与文档元数据	204 / 404 / 500 storage error
POST	/retrieve	只检索不生成，用于调试/评估	200; 422 参数错误; 502 Embedding provider 错误
POST	/ask	检索 + 证据门控 + LLM 回答	200; 无证据也是 200 拒答; 502 provider/model output 错误; 504 超时
GET	/health/live	进程存活	200
GET	/health/ready	本地依赖与必要配置就绪	200 / 503

11.1 /ask 响应结构

```
{
  "answer": "... [S1]",
  "sources": [
    {
      "source_id": "S1",
      "document_id": "...",
      "filename": "manual.pdf",
      "page": 12,
      "section": null,
      "chunk_index": 31,
      "excerpt": "..."
    }
  ],
  "latency_ms": {"retrieve": 42, "llm": 860, "total": 925},
  "request_id": "..."
}
```

11.2 统一错误响应

```
{
  "code": "UNSUPPORTED_DOCUMENT",
  "message": "Document cannot be parsed as supported text content",
  "details": null,
  "request_id": "..."}
}
```

- 422 用于请求/文件校验问题；409 用于重复文档；404 用于文档不存在。
- 外部模型/Embedding 服务异常转换为稳定业务错误，内部堆栈只写日志。
- DELETE 成功返回 204，不带 JSON 响应体。

12. 评估设计

评估是本项目的核心组成，不作为“以后再做”的增强项。固定数据集由同一批样例文档和 30 个问题组成：24 个可回答问题 + 6 个明确无答案问题。

指标	回答的问题	固定实现
Recall@K	正确来源是否进入 Top-K?	对 24 个可回答问题检查 retrieved metadata 是否命中 expected_document + expected_locator。
No-answer Accuracy	无资料问题是否被正确拒答?	6 个无答案问题应在 Evidence Gate 阶段拒答；同时统计可回答问题被误拒答的情况。
Latency	主要耗时在哪里?	分别记录 embed/retrieve、LLM、total；评测结果保存平均值与 P95（样本量允许时）。

实验原则

- 固定同一批文档和问题；实验过程中不为了结果好看而替换问题。
- 一次主要只改变一个变量；所有配置记录 embedding model、LLM model、chunk_size、overlap、Top-K、distance threshold。
- 检索失败与生成失败分开分析：Recall 先判断“证据有没有找到”，人工抽样再判断“模型有没有正确使用证据”。
- 结果文件保留失败 case 和原始配置，不只展示最优数字。

12.1 固定实验矩阵

实验	chunk_size	overlap	Top-K	目的
Baseline A	700	100	3	建立默认基线。
Chunk B	1000	150	3	只改变 chunk 大小，观察 Recall、噪声与延迟变化。
TopK C	700	100	5	只改变候选数量，观察 Recall 与上下文/LLM 延迟变化。

最终 README 记录三组实验的实际指标与结论，并据此选择项目默认配置；不要求某一种配置必然胜出。

13. 测试设计

测试层	重点	代表场景
Loader	格式、空文档、损坏/加密/纯图片 PDF、Markdown 编码	不支持文件返回受控错误；page/section metadata 正确。
Splitter	长度、overlap、边界、metadata 继承	PDF chunk 不跨页；Markdown section 不丢失；无空 chunk。
Metadata Repository	checksum UNIQUE、commit/rollback、delete	重复 checksum；模拟 commit 失败后 session 可继续。
Vector Store	add/query/delete、metadata filter	按 document_id 删除后不再命中；孤儿向量被上层过滤。
Retriever / Gate	Top-K、distance/rank、无证据判定	无关问题触发拒答；有证据问题不被误拒。
Context / Citation	S1/S2 分配、预算、未知标签	模型返回 [S9] 时结果不被映射为真实来源。
API	上传、列表、删除、retrieve、ask、非法输入、provider 异常	状态码和统一错误响应稳定；DELETE 204 无 body。
Evaluation	固定 questions、指标计算、结果落盘	同一配置重复运行可生成结构一致的实验结果。

测试完成标准

- pytest 自动化测试不调用真实云端模型；EmbeddingClient 与 LLMClient 使用 deterministic fake/mock。
- Chroma 测试使用临时持久化目录，SQLite 使用独立临时数据库；测试之间相互隔离。
- 至少覆盖成功、失败、边界、跨存储补偿、拒答与来源校验六类风险场景。
- 建议形成约 30-45 个核心测试；数量不是目标，关键是固定范围全部可验证。

14. 日志、健康检查与可观测性

能力	要求	示例字段 / 行为
请求日志	每个请求记录 request_id、方法、路径、状态码、总耗时	request_id, method, path, status_code, duration_ms
Ingestion 阶段	记录 parse/chunk/embed/vector_write 元数据与耗时，不记录全文	document_id, chunk_count, parse_ms, embed_ms, index_ms
Query 阶段	记录 retrieve/LLM/total；可记录 rank/distance 摘要	top_k, best_distance, retrieve_ms, llm_ms, total_ms
异常日志	未知异常记录 stack；provider/storage 记录阶段与 request_id	stage, exception_type, request_id
敏感信息	禁止记录 API Key、Authorization、完整文档、完整 Prompt/Context	必要时仅记录长度、hash 或 document_id。
liveness	只证明 FastAPI 进程可响应	GET /health/live -> 200
readiness	SQLite 可查询、Chroma 路径可读写、必要模型配置存在	不周期性调用收费 LLM/Embedding API；异常 -> 503

```
{"request_id": "a1b2...", "path": "/ask", "status_code": 200, "top_k": 3, "best_distance": 0.19, "retrieve_ms": 38, "llm_ms": 842, "total_ms": 901}
```

15. 运行、容器化与 CI

15.1 本地与 Docker 运行

- 本地学习阶段：Ubuntu / Python venv 运行 FastAPI；SQLite 与 Chroma 数据目录位于项目 `data/` 下。
- 容器阶段：Docker Compose 运行单个 api 服务，使用 named volume 持久化 `/app/data`；无需 MySQL、Redis 或独立 Chroma Server。
- 仓库提交 `.env.example`，不提交真实 API Key；模型名、Endpoint、Key、数据目录、`chunk/Top-K/threshold` 统一配置化。
- README 提供从 clone 到启动、上传样例、pytest、eval 的完整命令。

```
git clone <repo>
cp .env.example .env
docker compose up -d
docker compose exec api pytest
docker compose exec api python -m eval.run_eval
```

15.2 GitHub Actions CI

```
push / pull_request
-> checkout
-> setup Python
-> install dependencies
-> pytest
-> pass / fail
```

CI 不配置真实 LLM/Embedding API Key。所有依赖外部模型的测试通过 `fake client` 执行；真实模型只用于开发阶段手动运行固定评测，不把网络波动和调用费用引入持续集成。

15.3 仓库结构

```
docmind-rag/
├── app/
│   ├── main.py
│   ├── routers/
│   ├── ingestion/
│   │   ├── loader.py
│   │   ├── cleaner.py
│   │   └── splitter.py
│   ├── rag/
│   │   ├── retriever.py
│   │   ├── context.py
│   │   ├── prompt.py
│   │   └── pipeline.py
│   ├── storage/
│   │   ├── document_repo.py
│   │   └── vector_store.py
│   ├── clients/
│   │   ├── embeddings.py
│   │   └── llm.py
│   └── core/
├── tests/
├── eval/
│   ├── questions.jsonl
│   ├── run_eval.py
│   └── results/
├── data/sample/
├── .env.example
├── docker-compose.yml
└── README.md
```

16. 开发阶段与完成定义

阶段	建议时间	必须完成	完成定义
M1 骨架	第 1 周前半	FastAPI、配置、health、Client 接口、SQLite/Chroma 初始化	服务启动；/docs 正常；fake client 可注入。
M2 Ingestion	第 1 周后半	PDF/Markdown loader、cleaner、splitter、checksum、Embedding、索引	两种格式可建立索引；metadata/source locator 正确。
M3 Query	第 2 周	/retrieve、Retriever、Evidence Gate、Context、/ask、sources	能检索、拒答、回答并定位来源；未知引用被校验。
M4 管理与一致性	第 3 周	文档 list/detail/delete、重复检查、跨存储补偿	删除后不再检索；写入失败不产生可用脏文档。
M5 测试与日志	第 4 周	pytest、fake clients、request_id、阶段耗时日志	核心测试稳定通过；CI 不需真实 API Key。
M6 评估与交付	第 4-5 周	30 个固定问题、三组实验、Docker Compose、README、CI	可生成实验结果；新环境按文档启动、测试并复现实验流程。

17. 固定边界与设计取舍

问题	本项目固定取舍	原因 / 影响
权限	单用户私有/本地服务，无注册登录	项目重点是 RAG 质量链路；服务不应直接暴露为公网多租户系统。
PDF	只处理可提取文本 PDF，不做 OCR	避免 OCR/版面分析成为第二个项目主题。
向量库	Chroma 本地持久化	足以展示向量检索、 metadata 、 delete ；不虚构大规模并发需求。
检索	Dense vector search；无 Rewrite/Rerank/Hybrid	通过 chunk 与 Top-K 实验即可体现检索优化和评估方法。
生成	单次 2-step RAG；无 Agent	降低不可控步骤，便于定位检索失败与生成失败。
评估	Recall@K + no-answer accuracy + latency + 人工失败分析	不引入额外 LLM judge/Ragas，保持评估可解释和成本可控。
一致性	SQLite + Chroma 明确补偿，不宣称跨存储 ACID	符合实际技术边界，失败路径可测试。
模型	云端 API，通过 Client 抽象	项目展示 RAG 工程，而不是模型训练或特定供应商 SDK。

18. 最终验收清单

- ☐ 文本型 PDF 与 UTF-8 Markdown 均可成功摄取，非法/空/不可解析文件有稳定错误响应
- ☐ 每个 chunk 均携带 document_id、chunk_index 以及 PDF page 或 Markdown section 来源信息
- ☐ 重复 checksum 由数据库 UNIQUE 兜底，重复上传返回 409
- ☐ Chroma 索引与 SQLite Document 元数据保持固定写入顺序；失败补偿路径有自动化测试
- ☐ /retrieve 能返回 rank、distance、chunk 与来源 metadata，用于调试检索问题
- ☐ /ask 实现 Evidence Gate；无足够证据时不调用 LLM 并返回固定拒答
- ☐ 有证据回答使用 S1/S2...受控来源标签；未知 source label 不会被映射为真实来源
- ☐ 删除文档会删除其全部向量，删除后 /retrieve 与 /ask 不再使用旧内容
- ☐ 固定 30 个评测问题可重复运行，包含 24 个可回答问题和 6 个无答案问题
- ☐ 完成 Baseline、Chunk、Top-K 三组实验，并记录 Recall@K、拒答准确率与延迟
- ☐ pytest 覆盖 Loader、Splitter、Storage、Retriever、拒答、Citation、API 与失败补偿，CI 中无需真实模型 API
- ☐ 日志包含 request_id 与 parse/embed/retrieve/LLM/total 等关键耗时，且不打印 API Key、完整文档和完整 Prompt
- ☐ /health/live 与 /health/ready 行为正确，readiness 不周期性调用收费模型 API
- ☐ Docker Compose 可启动服务并持久化 SQLite/Chroma 数据；.env.example 可直接作为配置模板
- ☐ README 说明架构、数据流、API、运行、测试、评估、实验结果与固定设计取舍
- ☐ 公开仓库只包含可公开样例资料，不包含真实 API Key、私人文档或版权不明数据